Contextualized Word Embeddings

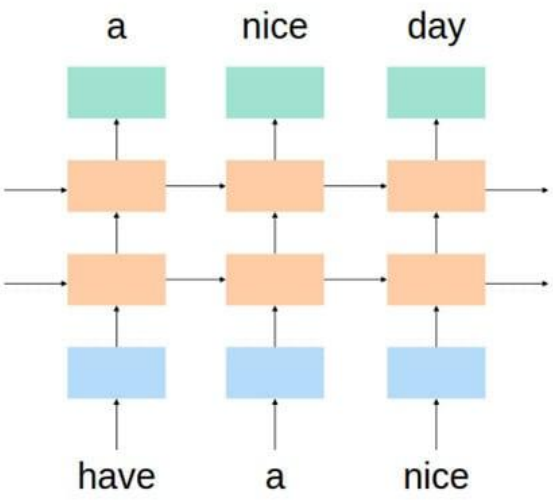
ELMo

ELMo: **E**mbeddings from **L**anguage **Mo**dels

Idea: contextualized word representations

Learn word vectors using long contexts instead of a context window

Learn a deep Bi-NLM and use all its layers in prediction



- ELMo (Embeddings from Language Models) architecture primarily uses **BiLSTM (Bidirectional Long Short-Term Memory)** networks for contextual word embeddings. However, it also incorporates **character-level CNNs** as part of its word representation process. Here's how both components work in ELMo:

1- Character-level CNNs

2- BiLSTM Layers:

1- Character-level CNNs:

- ELMo first represents words using character-level convolutional neural networks (CNNs).
- These CNNs extract morphological and subword features from characters, which help handle out-of-vocabulary words.
- The output from the CNN is then fed into the next layer.

2- BiLSTM Layers:

- The processed word representations are passed through a deep, two-layer **Bidirectional LSTM**.
- The BiLSTM captures rich contextual information by processing text in both forward and backward directions.
- The hidden states from different layers are combined to form the final **contextualized word embeddings**.

Character-level CNN

Character-level CNN

- Instead of treating words as atomic units (like Word2Vec), ELMo first represents each word at the **character level** using **CNNs (Convolutional Neural Networks)**.
- This allows it to capture **subword information**, making it **robust to out-of-vocabulary (OOV) words**.
- The CNN processes characters and applies different convolutional filters to extract useful features.
- Example: For the word "input_word", it is first broken down into individual characters:

['i', 'n', 'p', 'u', 't', '_', 'w', 'o', 'r', 'd']

Then, different **CNN kernels** (e.g., of sizes **1, 2, 3, 4, 5, 6, 7**) extract n-gram-like character features, followed by max-pooling and concatenation.

- The original ELMo model uses the following convolutional kernel sizes and channels:
  - Kernel sizes: **1, 2, 3, 4, 5, 6, 7**
  - Number of filters per kernel: **32, 32, 64, 128, 256, 512, 1024**
  - Final output vector size: **2048-dimensional**

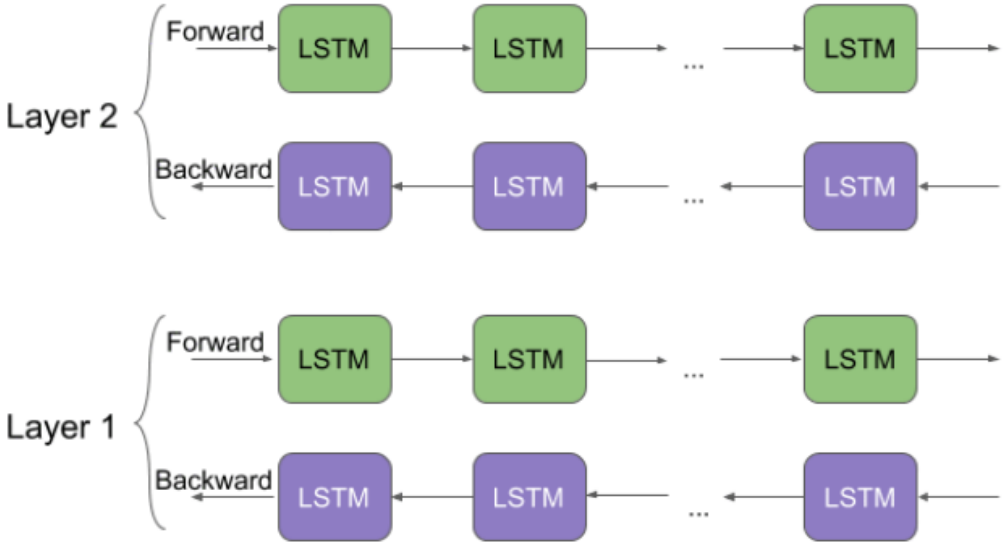
Bidirectional LSTM (BiLSTM)

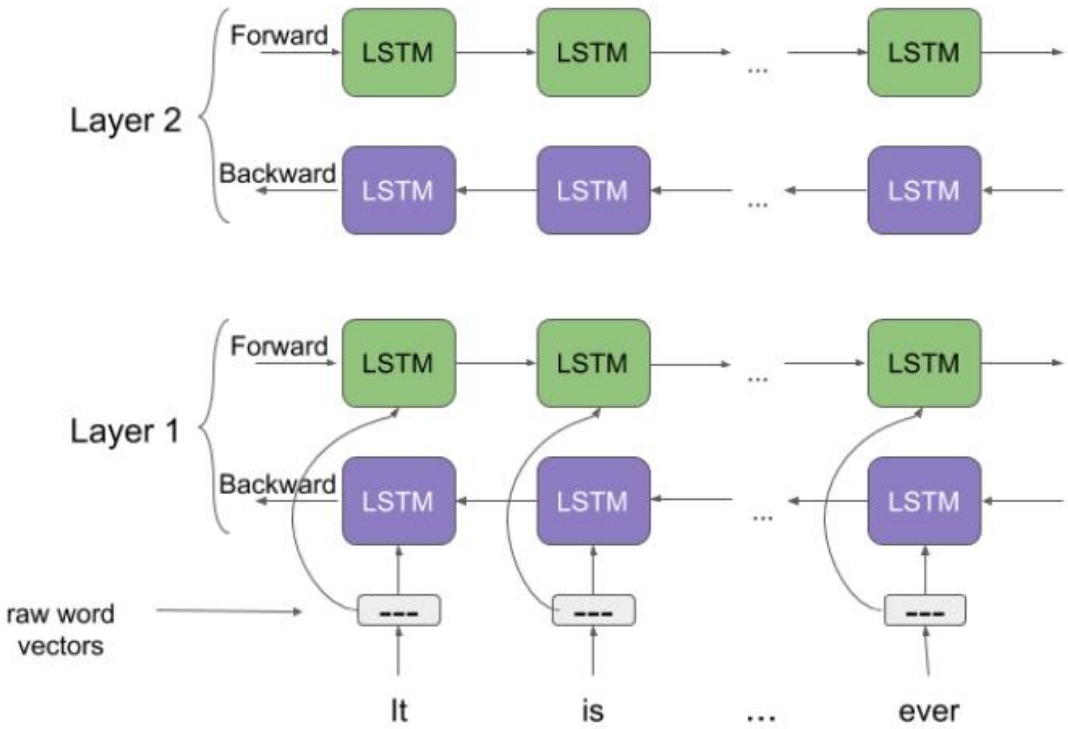
- After extracting character features, the processed word representations are passed through a **two-layer BiLSTM**.
- Unlike standard BiLSTMs, ELMo **separates the forward and backward LSTM connections** before merging them.
- **Why BiLSTM?**
- Standard LSTMs process sequences only in a **left-to-right** (forward) or **right-to-left** (backward) manner.
- A **BiLSTM** captures both **preceding and succeeding context** for each word.

- Example: Consider the sentence
- "The cat sat on the mat"
- For the word "sat",
- The **forward LSTM** sees "The cat" before it.
- The **backward LSTM** sees "on the mat" after it.
- This bidirectional context allows "sat" to get a **better representation** depending on the sentence.

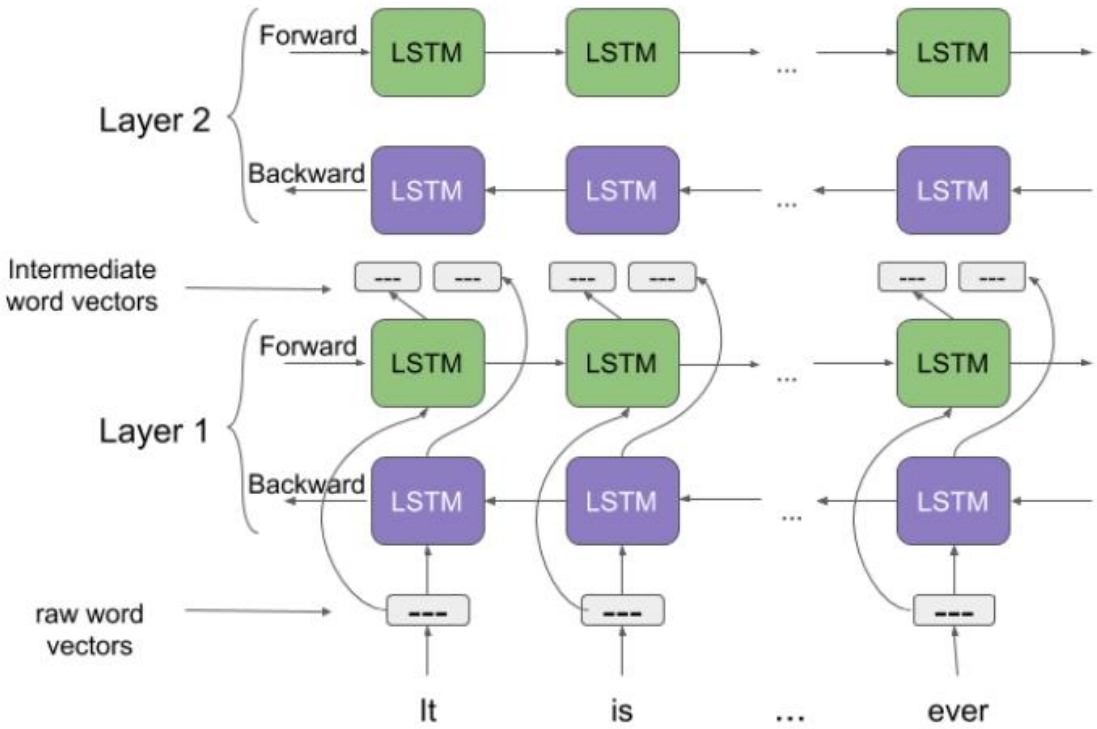
- **Layer-wise Features:**
- The **first BiLSTM layer** captures lower-level syntactic information (e.g., POS tagging).
- The **second BiLSTM layer** captures higher-level semantic information (e.g., word-sense disambiguation).

- After obtaining contextualized embeddings from BiLM, a **task-specific layer** is added to fine-tune embeddings for different NLP tasks.
- **Weighted Sum of BiLM Outputs**
  Instead of using just the top BiLSTM layer, ELMo allows a weighted sum of **all layers' outputs** to be learned dynamically.
- **Scaling and Normalization**
  The weighted sum is scaled before passing to the final **task-specific model**.

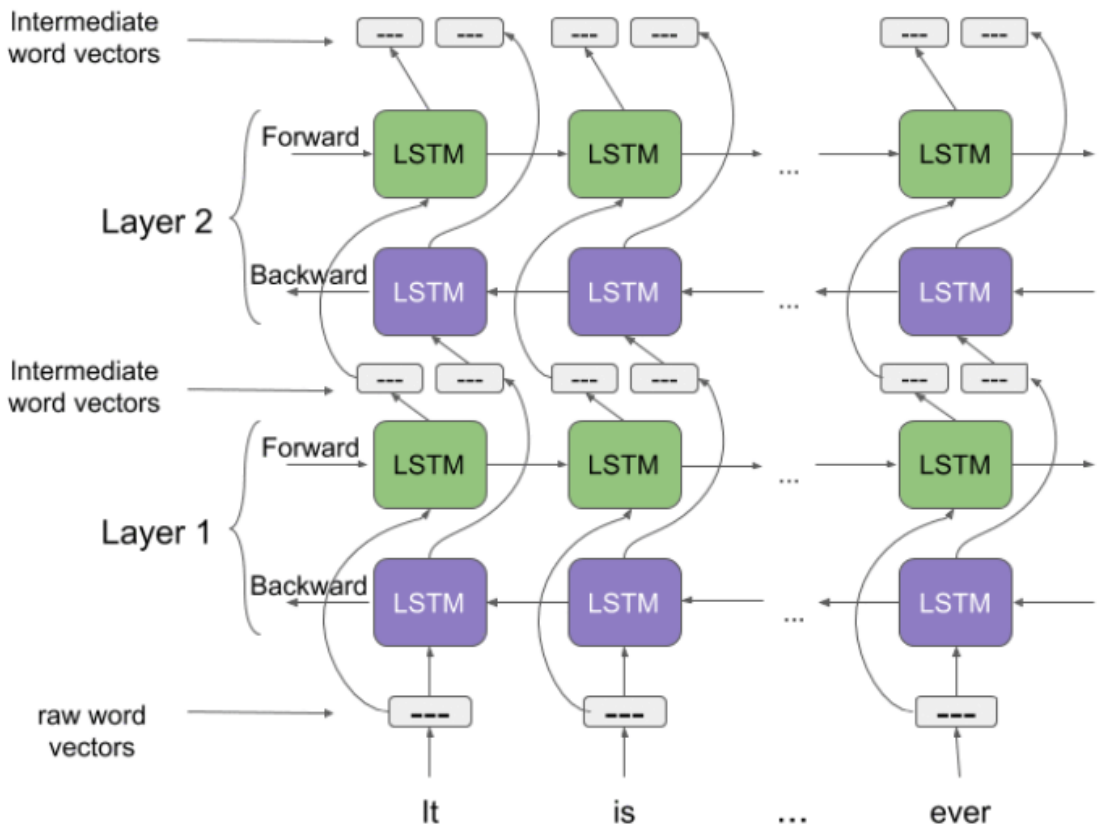

Layer 2
Forward
LSTM
LSTM
...
LSTM
Backward
LSTM
LSTM
...
LSTM
Layer 1
Forward
LSTM
LSTM
...
LSTM
Backward
LSTM
LSTM
...
LSTM

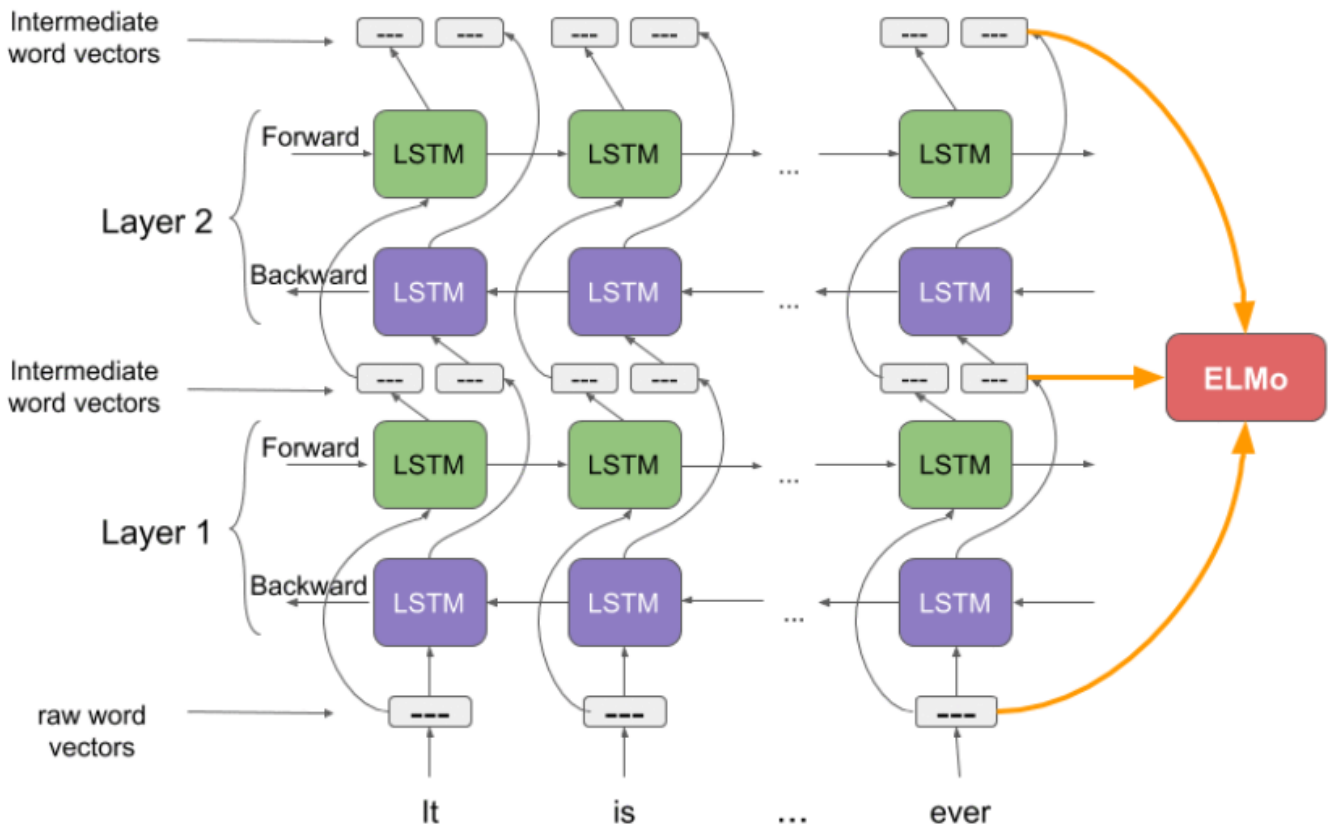

Layer 2
Forward
LSTM
LSTM
...
LSTM
Backward
LSTM
LSTM
...
LSTM
Layer 1
Forward
LSTM
LSTM
...
LSTM
Backward
LSTM
LSTM
...
LSTM
raw word vectors

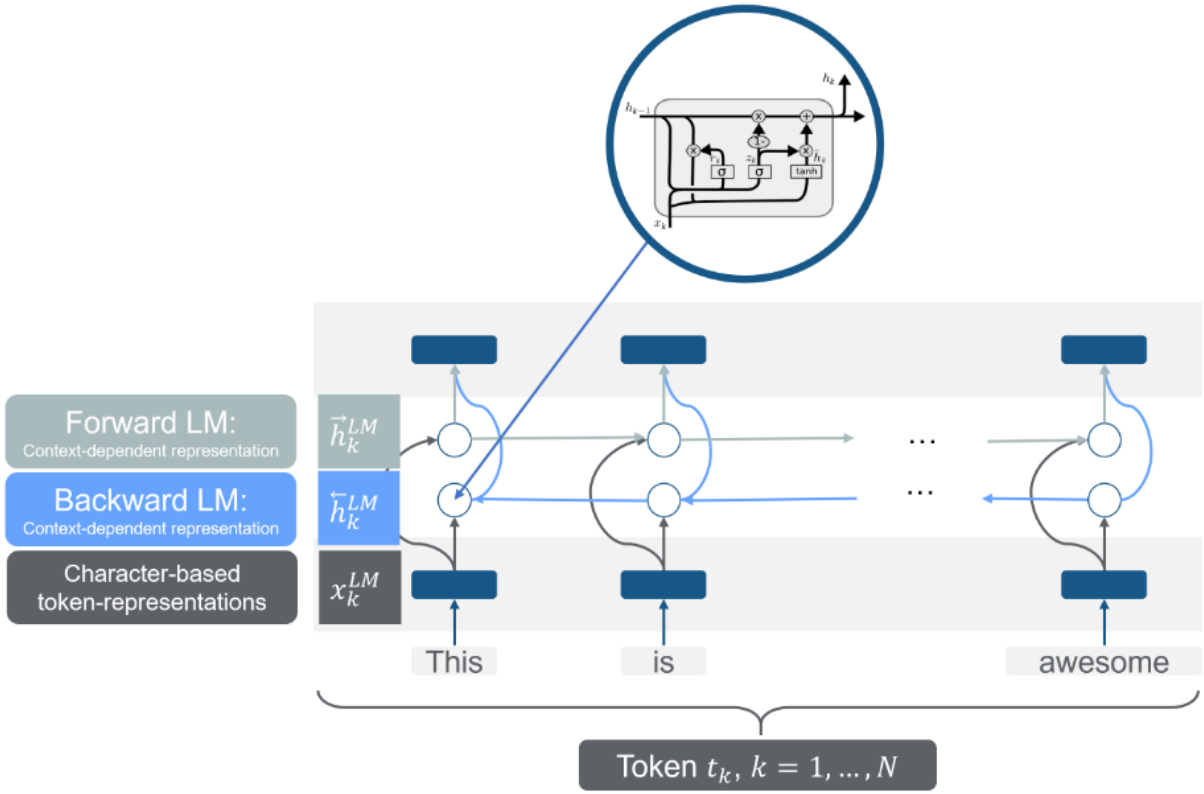
It
is
...
ever


Forward
LSTM
LSTM
...
LSTM
Layer 2
Backward
LSTM
LSTM
...
LSTM
Intermediate
word vectors

Forward
LSTM
LSTM
...
LSTM
Layer 1
Backward
LSTM
LSTM
...
LSTM
raw word
vectors
It
is
...
ever


Intermediate word vectors
Forward
LSTM
Layer 2
Backward
Intermediate word vectors
Forward
Layer 1
Backward
raw word vectors
It
is
...
ever


Intermediate word vectors
Forward
LSTM
Layer 2
Backward
Intermediate word vectors
Forward
Layer 1
Backward
raw word vectors
ELMo
It
is
...
ever



Source: Carolin Becker

```
ELMoBiLM(
  (_elmo_char_encoder): ELMoCharacterEncoder(
    (_char_embedding): Embedding(262 -> 16, float32)
    (_convolutions): ConvolutionalEncoder(
      (_convs): HybridConcurrent(
        (0): HybridSequential(
          (0): Conv1D(16 -> 32, kernel_size=(1,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
        (1): HybridSequential(
          (0): Conv1D(16 -> 32, kernel_size=(2,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
        (2): HybridSequential(
          (0): Conv1D(16 -> 64, kernel_size=(3,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
        (3): HybridSequential(
          (0): Conv1D(16 -> 128, kernel_size=(4,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
        (4): HybridSequential(
          (0): Conv1D(16 -> 256, kernel_size=(5,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
        (5): HybridSequential(
          (0): Conv1D(16 -> 512, kernel_size=(6,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
        (6): HybridSequential(
          (0): Conv1D(16 -> 1024, kernel_size=(7,), stride=(1,))
          (1): HybridLambda(<lambda>)
          (2): Activation(relu)
        )
      )
      (_highways): Highway(
        (hnet): HybridSequential(
          (0): Dense(2048 -> 4096, linear)
```

```
            )
            (_activation): Activation(relu)
          )
          (_projection): Dense(2048 -> 128, linear)
        )
      )
      (_elmo_lstm): BiLMEncoder(
        (forward_layers): HybridSequentialRNNCell(
        (0): HybridSequentialRNNCell(
          (0): LSTMPCellWithClip(128 -> 4096 -> 128)
          )
        (1): HybridSequentialRNNCell(
          (0): ResidualCell(LSTMPCellWithClip(128 -> 4096 -> 128))
          )
        )
        (backward_layers): HybridSequentialRNNCell(
        (0): HybridSequentialRNNCell(
          (0): LSTMPCellWithClip(128 -> 4096 -> 128)
          )
        (1): HybridSequentialRNNCell(
          (0): ResidualCell(LSTMPCellWithClip(128 -> 4096 -> 128))
          )
        )
      )
    )
```

- We will use:

  "Dog“

Step 1: Character Tokenization

- Instead of treating "Dog" as a whole word, we break it into characters:

['<bow>', 'D', 'o', 'g', '<eow>']

<bow> = Beginning-of-word marker

<eow> = End-of-word marker

- So the input is:

  <bow> D o g <eow>

Step 2: Character Embeddings

Each character is mapped to a numeric vector. Suppose the embedding size is **4**.

Character	Embedding (size=4)
<bow>	[0.1, 0.2, 0.3, 0.4]
D	[0.5, 0.6, 0.7, 0.8]
o	[0.2, 0.3, 0.5, 0.7]
g	[0.9, 0.4, 0.2, 0.1]
<eow>	[0.3, 0.6, 0.8, 0.5]

- So, the word "**Dog**" is now represented as:

```
[
  [0.1, 0.2, 0.3, 0.4],   # <bow>
  [0.5, 0.6, 0.7, 0.8],   # D
  [0.2, 0.3, 0.5, 0.7],   # o
  [0.9, 0.4, 0.2, 0.1],   # g
  [0.3, 0.6, 0.8, 0.5]    # <eow>
]
```

This is a **(5 × 4) matrix** (5 characters, 4 features per character).

Step 3: Applying CNN

- We apply a **1D convolution** with a **filter of size 3** (sliding over 3 characters at a time). The filter has **weights** and a **bias**.
- Assume a **single filter** with these weights (random example):

```
W = [
  [0.1, 0.2, 0.3, 0.4],  # For first character
  [0.5, 0.6, 0.7, 0.8],  # For second character
  [0.9, 1.0, 1.1, 1.2]   # For third character
]
```

and **bias** b = 0.1.

- **Convolution Operation**

We slide the filter across the input:

1- Apply filter to <bow> D o

(0.1*0.1) + (0.2*0.2) + (0.3*0.3) + (0.4*0.4) + (0.5*0.5) + (0.6*0.6) + (0.7*0.7) + (0.8*0.8) + (0.9*0.2) + (1.0*0.3) + (1.1*0.5) + (1.2*0.7) + 0.1

→ Output **(some value, let's call it $x1$)**.

2- Apply filter to D o g

(0.1*0.5) + (0.2*0.6) + ... + (1.2*0.1) + 0.1

→ Output $\boldsymbol{x}\mathbf{2}$.

3- Apply filter to o g <eow>

(0.1*0.2) + (0.2*0.3) + ... + (1.2*0.5) + 0.1

→ Output $\boldsymbol{x}\mathbf{3}$.

- Now we have a new feature vector $[x1, x2, x3]$.

Step 4: Max Pooling

After convolution, we apply **max pooling** to pick the **strongest feature**.

$$[x1 = 0.5, x2 = 0.8, x3 = 0.3]$$

Then **Max Pooling** selects: 0.8

Step 5: Feeding into BiLSTM

- Now, the **word representation** [0.8] is passed into a **Bidirectional LSTM** (BiLSTM).
- This helps understand **context** by processing neighboring words.

Final Representation

- The final **ELMo word embedding** for "Dog" is computed as:
- ELMo = α1 * CNN_output + α2 * BiLSTM1_output + α3 * BiLSTM2_output
- where **α1, α2, α3** are learned weights.

- **Character Tokenization** → ['<bow>', 'D', 'o', 'g', '<eow>']
- **Character Embeddings** → Each character converted to numbers.
- **Character CNN** → Detects patterns like "D+o" or "o+g".
- **Max Pooling** → Keeps the strongest features.
- **BiLSTM** → Adds context from surrounding words.
- **Final ELMo Embedding** → A mix of CNN & LSTM outputs.

Model	Pretrained?	Type	Contextual?	Explanation
Word2Vec	☑ Yes	Static Embedding	✕ No	Words are mapped to fixed vectors (e.g., "bank" has the same meaning in "river bank" and "bank account").
GloVe	☑ Yes	Static Embedding	✕ No	Similar to Word2Vec, trained using word co-occurrence statistics.
FastText	☑ Yes	Static + Subword	✕ No	Improves on Word2Vec/GloVe by considering **subword units**, useful for rare words.
RNN	✕ No	Neural Network	☑ Yes	RNNs process sequences but do **not** have pretrained versions by default.
LSTM	✕ No	Neural Network	☑ Yes	A specialized type of RNN; needs training on specific data. No direct pretrained version.
ELMo	☑ Yes	Contextual Embedding	☑ Yes	Uses **pretrained** bi-directional LSTMs. Each word's vector **depends on its context** in a sentence.